CLASSIFICATION

INTRODUCTION

- Task of assigning objects to one of several predefined categories
- Input data for a classification task is a collection of records

Given a collection of records (training set):

- Each record is by characterized by a tuple (x,y) where x is the attribute set and y is the class label

- x: attribute, predictor, independent variable, input
- y: class, response, dependent variable, output

INTRODUCTION

- Example: Sample data set used for classifying vertebrates
- Attribute set includes properties of a vertebrate such as its body temperature, skin cover, method of reproduction, ability to fly, and ability to live in water
- Class label must be a discrete attribute

Task	Attribute set, x	Class label, y
Categorizing email messages	Features extracted from email message header and content	spam or non-spam
Cataloging galaxies	Features extracted from telescope images	Elliptical, spiral, or irregular-shaped galaxies

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hiber-nates	Class Label
human	warm-blooded	hair	yes	no	no	yes	no	mammal
python	cold-blooded	scales	no	no	no	no	yes	reptile
salmon	cold-blooded	scales	no	yes	no	no	no	fish
whale	warm-blooded	hair	yes	yes	no	no	no	mammal
frog	cold-blooded	none	no	semi	no	yes	yes	amphibian
komodo dragon	cold-blooded	scales	no	no	no	yes	no	reptile
bat	warm-blooded	hair	yes	no	yes	yes	yes	mammal
pigeon	warm-blooded	feathers	no	no	yes	yes	no	bird
cat	warm-blooded	fur	yes	no	no	yes	no	mammal
leopard shark	cold-blooded	scales	yes	yes	no	no	no	fish
turtle	cold-blooded	scales	no	semi	no	yes	no	reptile
penguin	warm-blooded	feathers	no	semi	no	yes	no	bird
porcupine	warm-blooded	quills	yes	no	no	yes	yes	mammal
eel	cold-blooded	scales	no	yes	no	no	no	fish
salamander	cold-blooded	none	no	semi	no	yes	yes	amphibian

- Task of learning a target function f that maps each attribute set x to one of the predefined class labels y

- Target function is also known informally as a classification model

Classification model is useful for the following purposes:

- **Descriptive Modeling**
  - It can serve as an explanatory tool to distinguish between objects of different classes

- **Predictive Modeling**
  - It can be used to predict the class label of unknown records
  - It can be treated as a black box that automatically assigns a class label when presented with the attribute set of an unknown record

Name	Body Temperature	Skin Cover	Gives Birth	Aquatic Creature	Aerial Creature	Has Legs	Hiber-nates	Class Label
gila monster	cold-blooded	scales	no	no	no	yes	yes	?

General Approach to Solving a Classification Problem

- A classification technique (or classifier) is a systematic approach to building classification models from an input data set

- Examples: Decision tree classifiers, Rule-based classifiers, Neural Networks, Support Vector Machines, Naive Bayes classifiers

- Each technique employs a learning algorithm to identify a model that best fits the relationship between the attribute set and class label of the input data

- The model generated by a learning algorithm should both fit the input data well and correctly predict the class labels of records it has never seen before

- Key objective of the learning algorithm is to build models with good generalization capability (models that accurately predict the class labels of previously unknown records)

- First, a training set consisting of records whose class labels are known must be provided

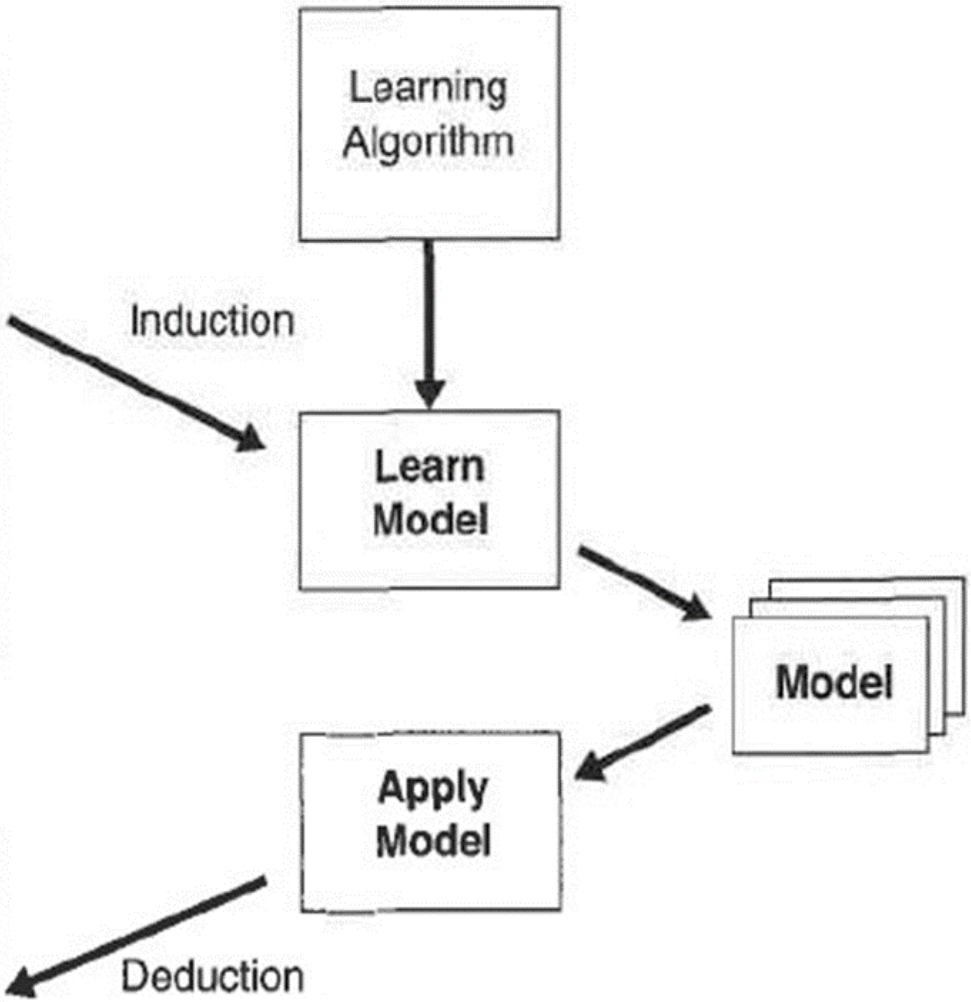
- The training set is used to build a classification model which is subsequently applied to the test set, which consists of records with unknown class labels

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Test Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?



- Evaluation of the performance of a classification model is based on the counts of test records correctly and incorrectly predicted by the model

- Counts are tabulated in a table known as a **Confusion matrix**

		Predicted Class	
		$Class = 1$	$Class = 0$
Actual Class	$Class = 1$	f_{11}	f_{10}
	$Class = 0$	f_{01}	f_{00}

- Each entry f_{ij} in this table denotes the number of records from class i predicted to be of class j
- f_{01} is the number of records from class 0 incorrectly predicted as class 1
- Total number of correct predictions made by the model is $(f_{11} + f_{00})$
- Total number of incorrect predictions is $(f_{01} + f_{10})$

Performance metrics

- Accuracy

  =No. of correct predictions / Total No. of predictions

  = $(f_{11}+f_{00})/(f_{11}+f_{10}+f_{01}+f_{00})$

- Error rate

  =No. of wrong predictions / Total No. of predictions

  = $(f_{01}+f_{10})/(f_{11}+f_{10}+f_{01}+f_{00})$

Decision Tree Classifier

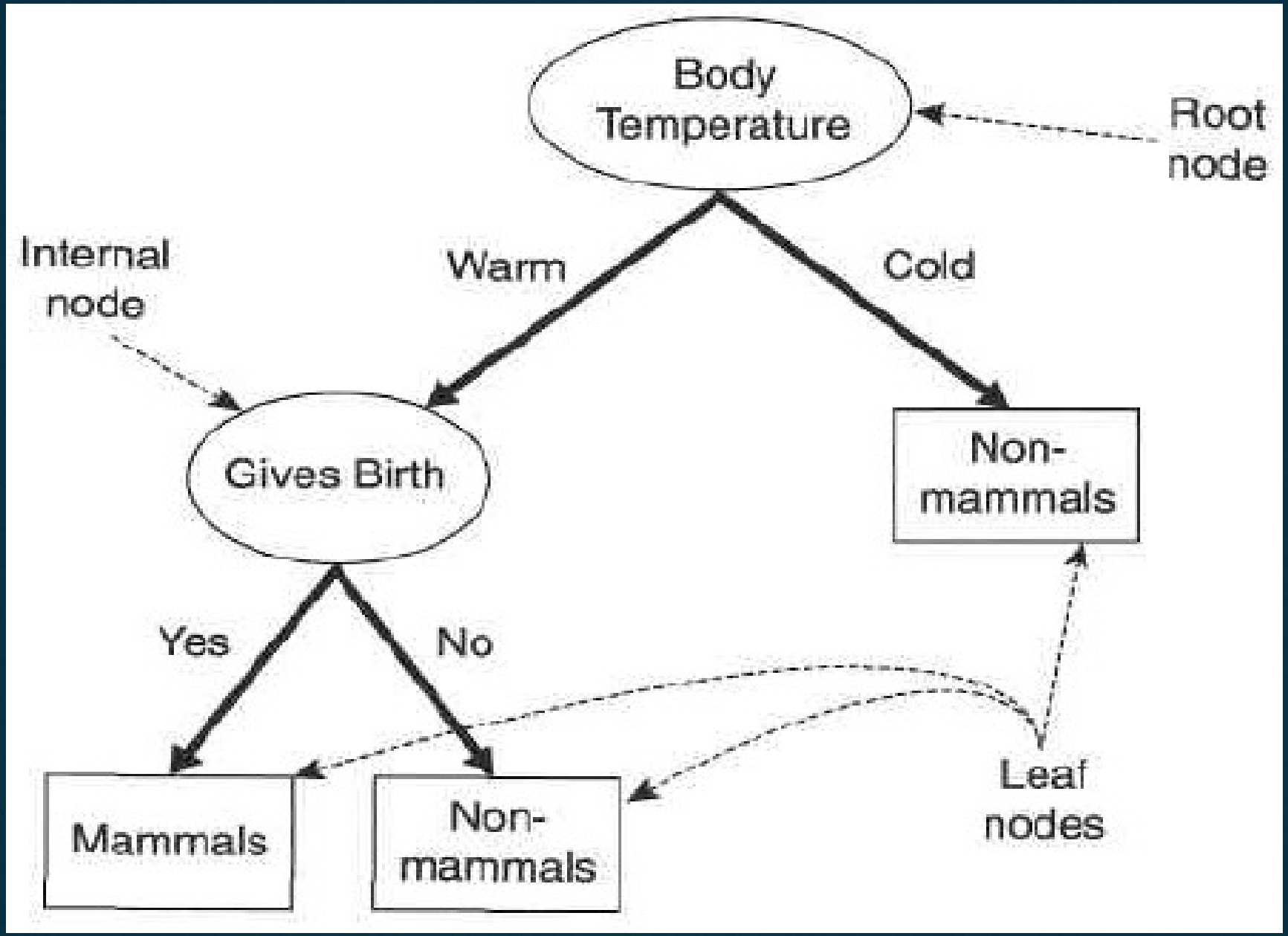
How a Decision Tree Works:

- Solves a classification problem by asking a series of carefully crafted questions about the attributes of the test record
- Each time we receive an answer, a follow-up question is asked until we reach a conclusion about the class label of the record
- Series of questions and their possible answers can be organized in the form of a decision tree
- Hierarchical structure consisting of nodes and directed edge

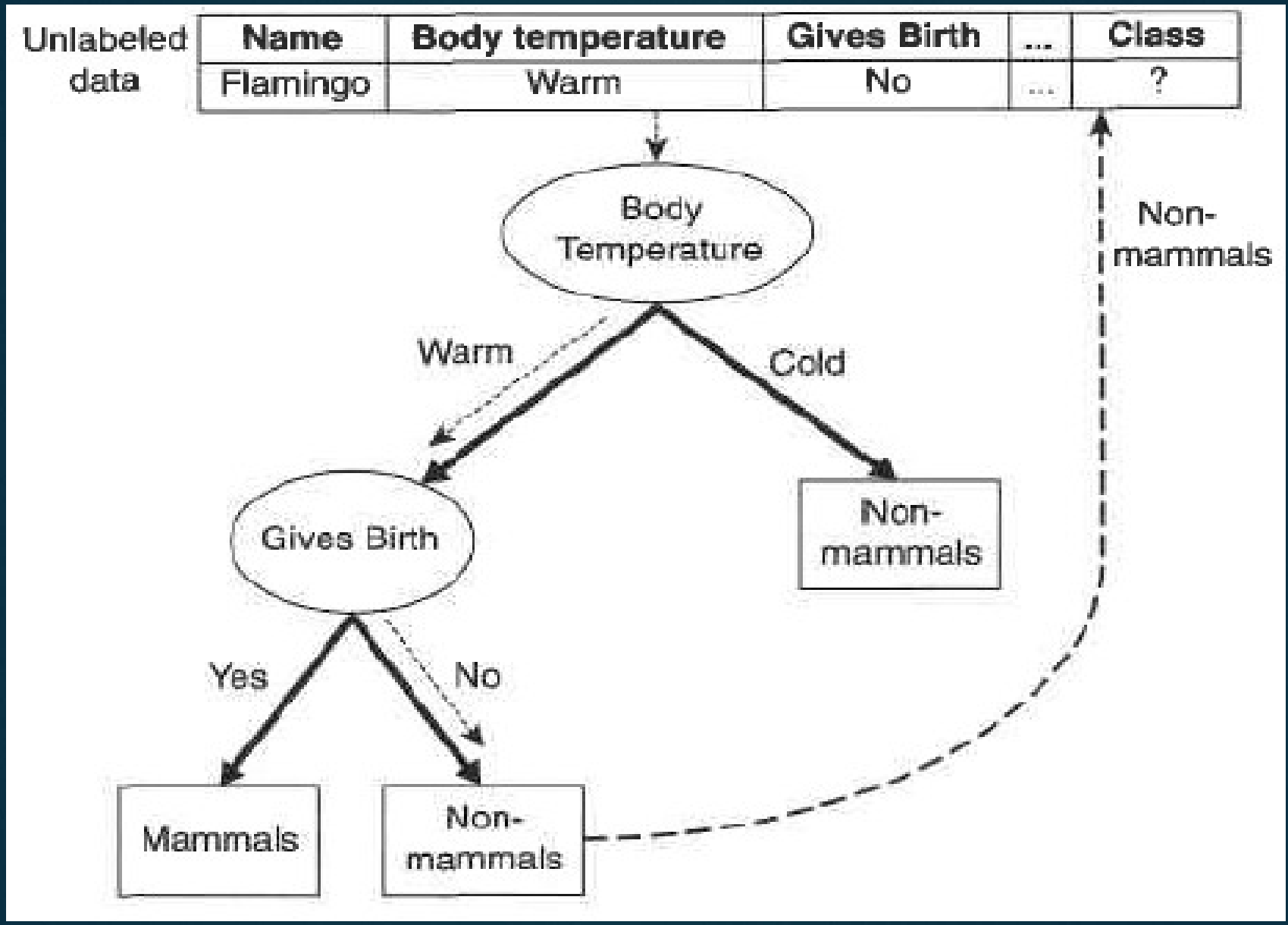
Three types of nodes:

- Root node that has no incoming edges and zero or more outgoing edge
- Internal nodes each of which has exactly one incoming edge and two or more outgoing edge
- Leaf or terminal nodes, each of which has exactly one incoming edge and no outgoing edge

- Each leaf node is assigned a *class label*
- Non-terminal nodes, which include the root and other internal nodes, contain *attribute test conditions* to separate records that have different characteristics


Body Temperature
Root node
Internal node
Warm
Cold
Gives Birth
Non-mammals
Yes
No
Mammals
Non-mammals
Leaf nodes

- Classifying a test record is straightforward once a decision tree has been constructed
- Starting from the root node, apply the test condition to the record and follow the appropriate branch based on the outcome of the test
- This will lead either to another internal node, for which a new test condition is applied, or to a leaf node
- The class label associated with the leaf node is then assigned to the record


Unlabeled data
Name
Body temperature
Gives Birth
...
Class
Flamingo
Warm
No
...
?
Body Temperature
Warm
Cold
Gives Birth
Non-mammals
Yes
No
Mammals
Non-mammals
Non-mammals

How to Build a Decision Tree:

- Finding the optimal tree is computationally infeasible because of the exponential size of the search space
- Efficient algorithms have been developed to induce a reasonably accurate decision tree in a reasonable amount of time

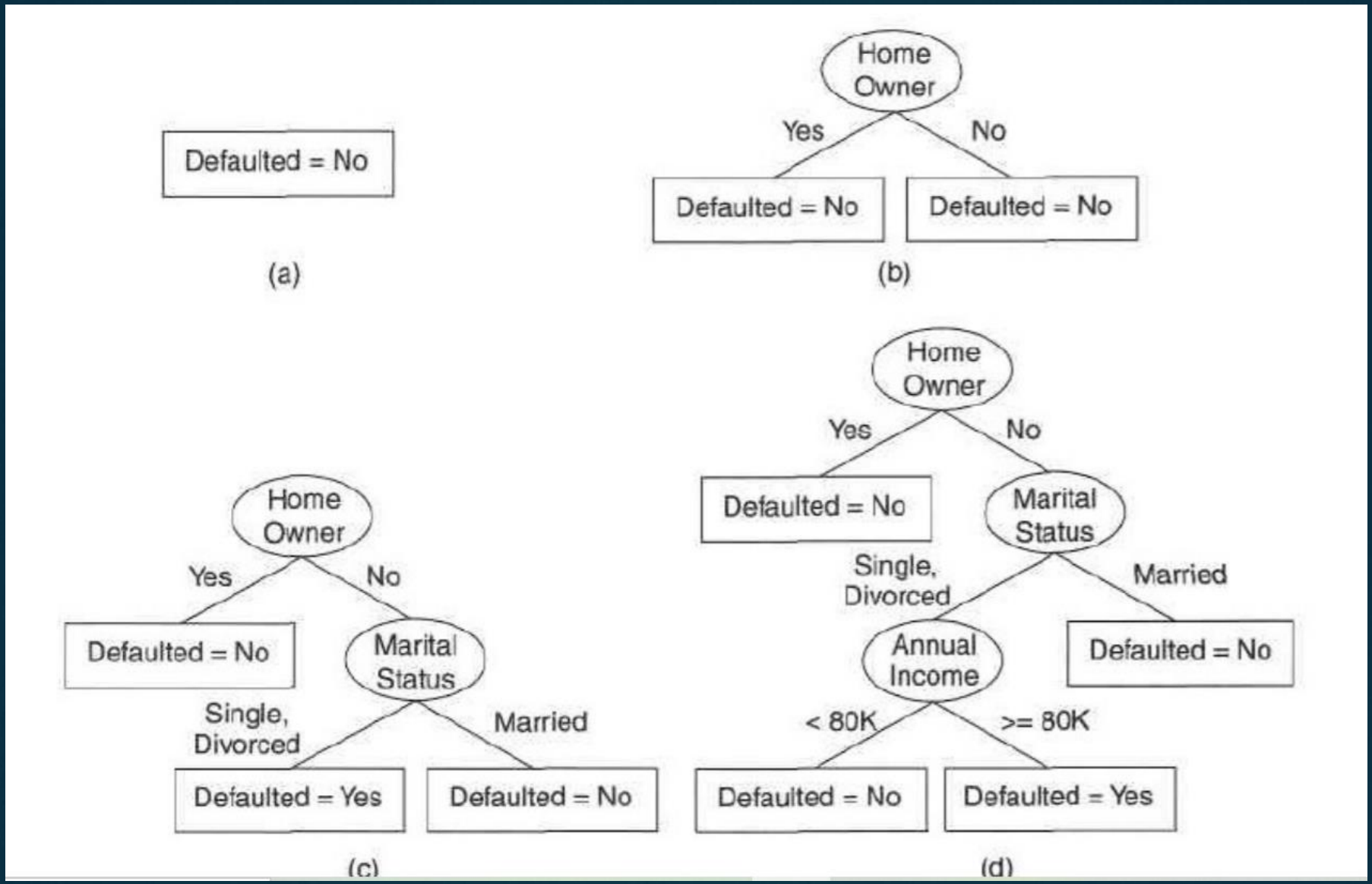
Hunt's Algorithm:

- A decision tree is grown in a recursive fashion by partitioning the training records into successively purer subsets
- Let D_t be the set of training records that are associated with node t
- $y = \{y_1, y_2, \ldots . y_c\}$ be the class labels

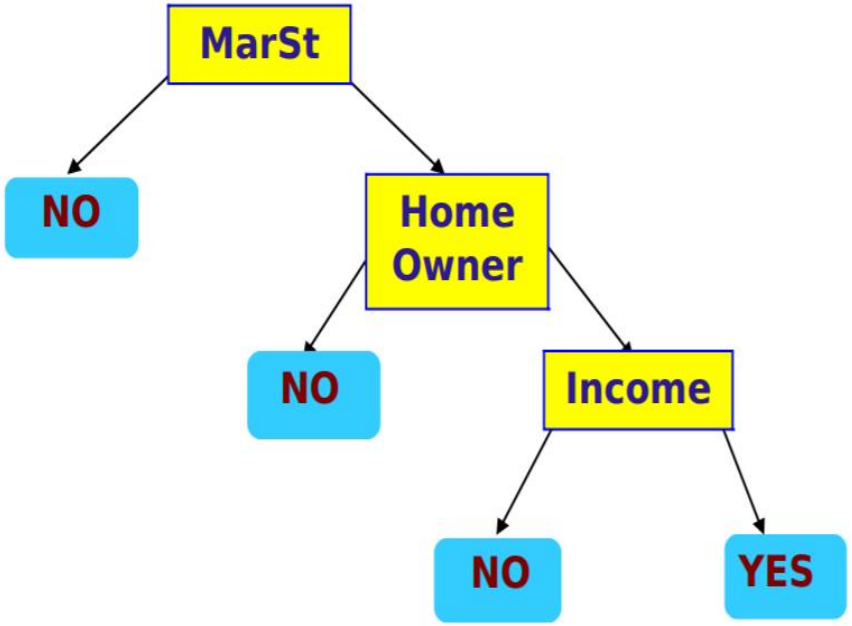
Recursive definition of Hunt's algorithm

- Step 1: If all the records in D_t belong to the same class y_t , then t is a leaf node labelled as y_t
- Step 2: If D_t contains records that belong to more than one class, an attribute test condition is selected to partition the records into smaller subsets
- A child node is created for each outcome of the test condition and the records in D_t are distributed to the children based on the outcomes
- The algorithm is then recursively applied to each child node

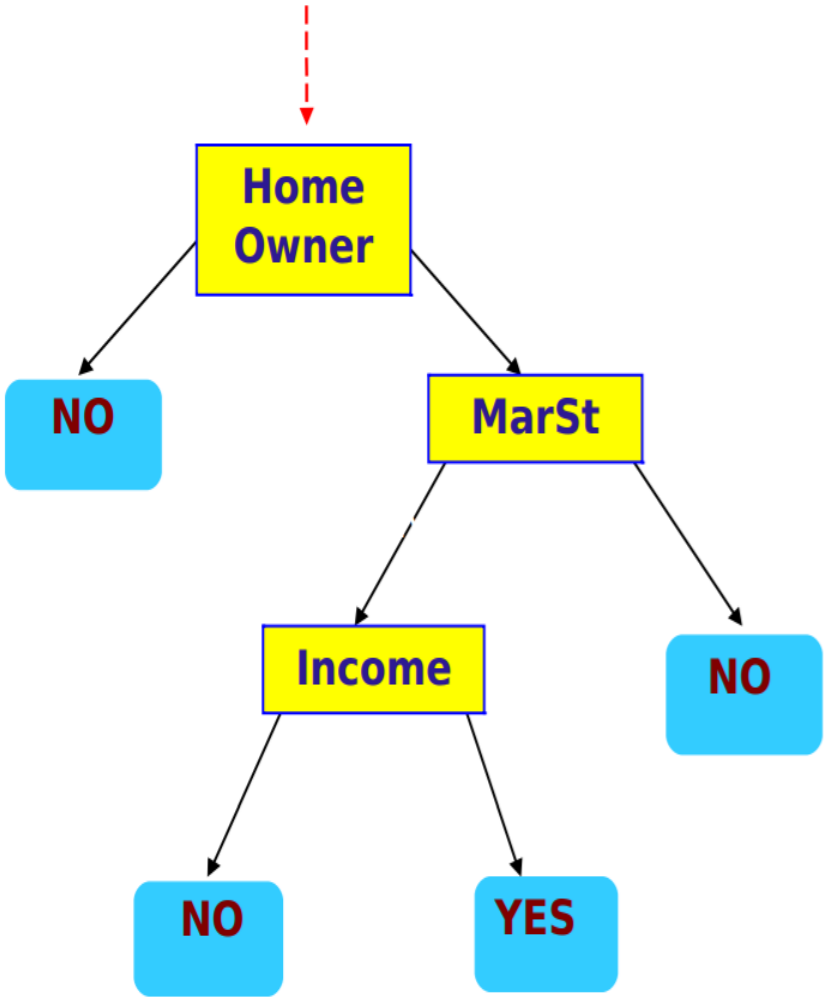
	binary	categorical	continuous	class
Tid	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes


Defaulted = No
(a)
Home Owner
Yes
No
Defaulted = No
Defaulted = No
(b)
Home Owner
Yes
No
Defaulted = No
Marital Status
Single, Divorced
Married
Annual Income
Defaulted = No
< 80K
>= 80K
Defaulted = No
Defaulted = Yes
(d)
Home Owner
Yes
No
Defaulted = No
Marital Status
Single, Divorced
Married
Defaulted = Yes
Defaulted = No
(c)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	categorical	*categorical*	*continuous*	*class*
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

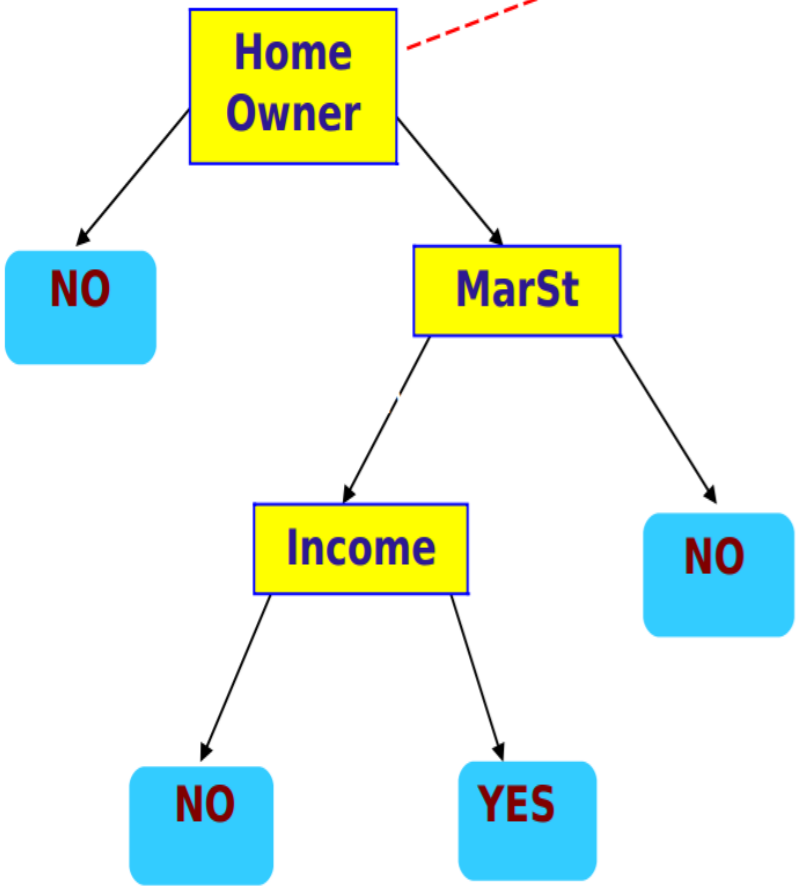


Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

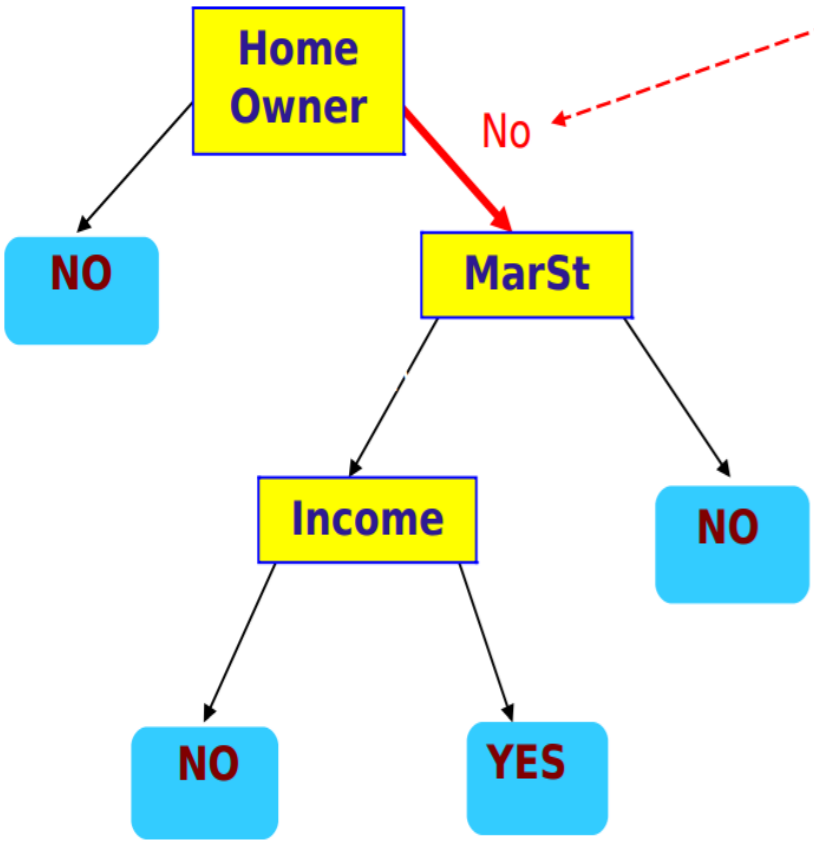
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



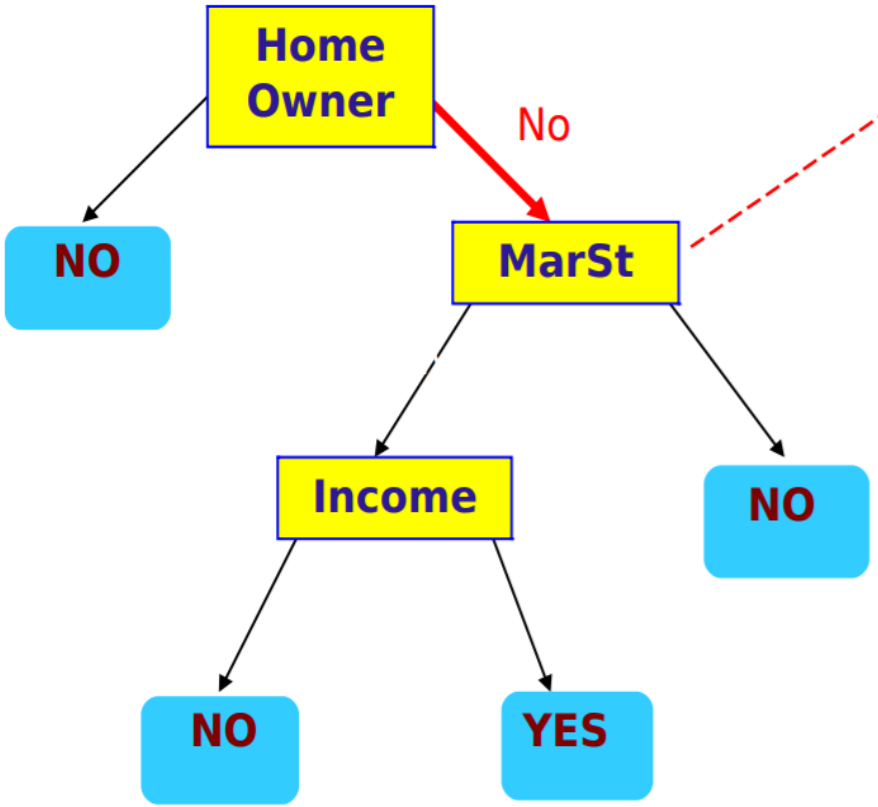
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



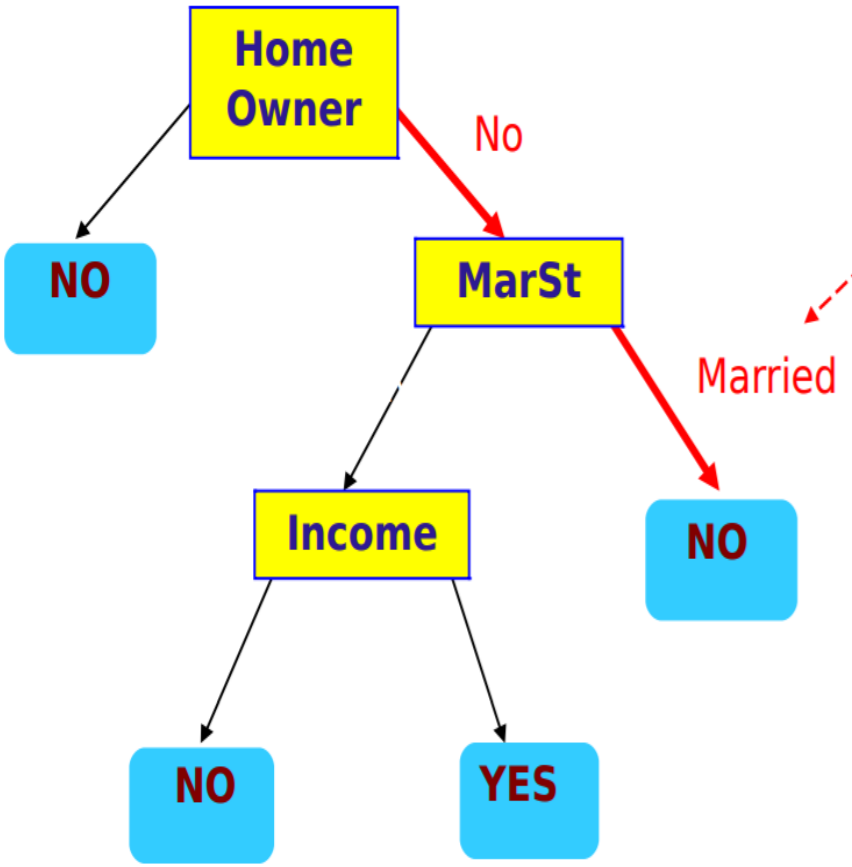
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



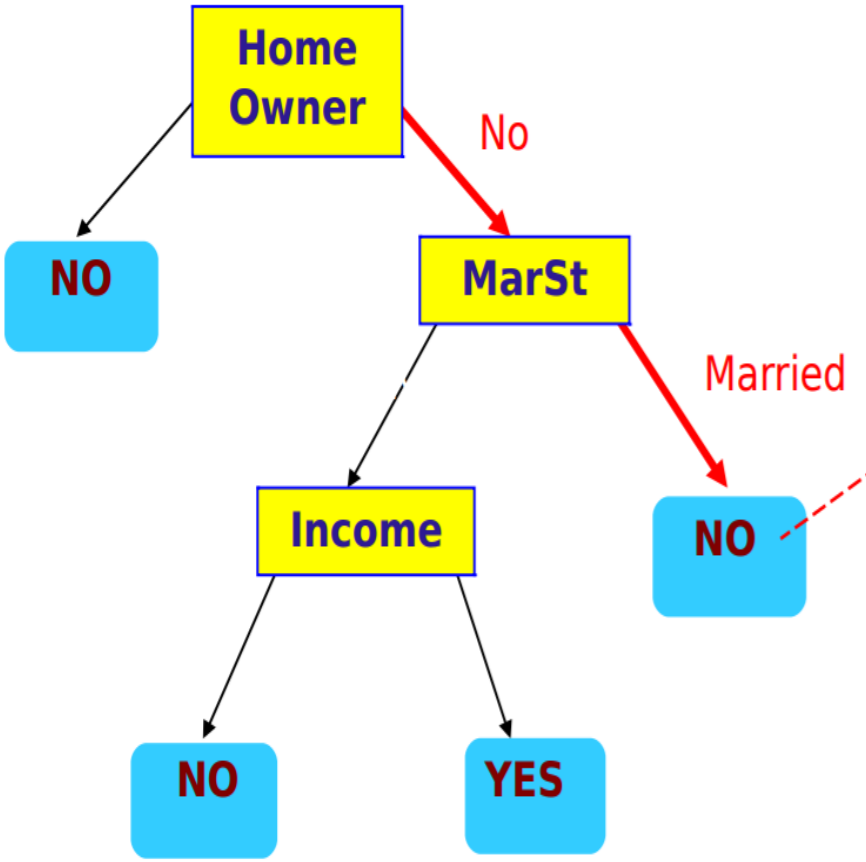
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



- Hunt's algorithm will work if every combination of attribute values is present in the training data and each combination has a unique class label
- These assumptions are too stringent for use in most practical situations

- Additional conditions are needed to handle the following cases:
- It is possible for some of the child nodes created in Step 2 to be empty; i.e., there are no records associated with these nodes. In this case the node is declared a leaf node with the same class label as the majority class of training records associated with its parent node
- In Step 2, if all the records associated with D; have identical attribute values (except for the class label), then it is not possible to split these records any further. In this case, the node is declared a leaf node with the same class label as the majority class of training records associated with this node

- Additional conditions are needed to handle the following cases:
- It is possible for some of the child nodes created in Step 2 to be empty; i.e., there are no records associated with these nodes. In this case the node is declared a leaf node with the same class label as the majority class of training records associated with its parent node
- In Step 2, if all the records associated with D; have identical attribute values (except for the class label), then it is not possible to split these records any further. In this case, the node is declared a leaf node with the same class label as the majority class of training records associated with this node

Design Issues of Decision Tree Induction

- How should training records be split?
  - Method for expressing test condition
    - depending on attribute types
  - Measure for evaluating the goodness of a test condition

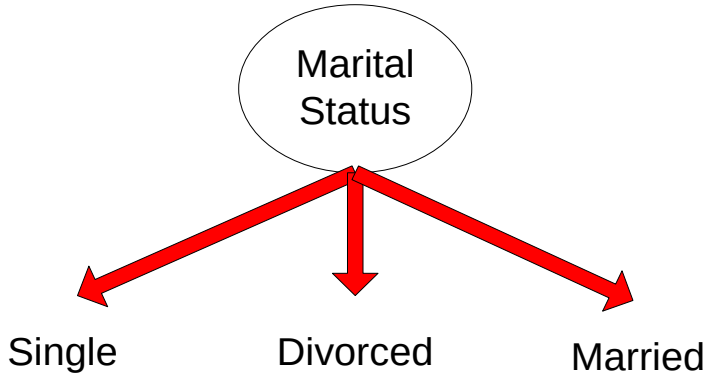
- How should the splitting procedure stop?
  - Stop splitting if all the records belong to the same class or have identical attribute values
  - Early termination

Methods for Expressing Test Conditions

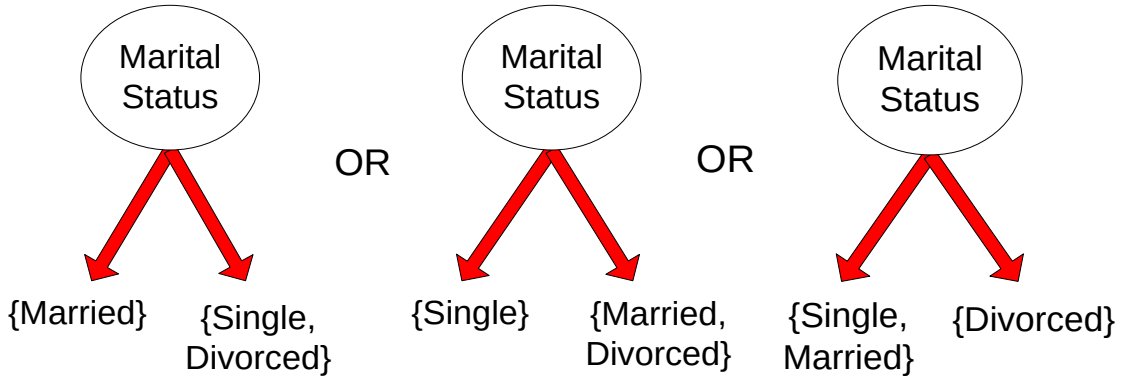
- Depends on attribute types
  - Binary
  - Nominal
  - Ordinal
  - Continuous

Test Condition for Nominal Attributes

- Multi-way split:
  - Use as many partitions as distinct values.

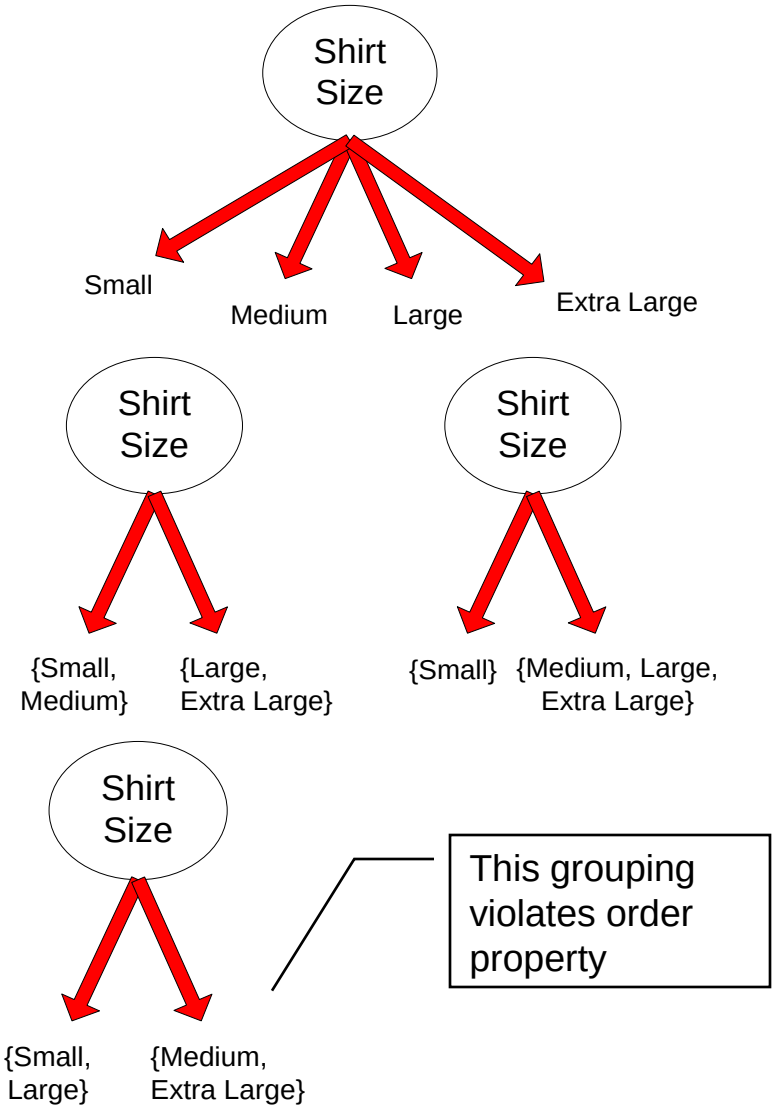


- Binary split:
  - Divides values into two subsets

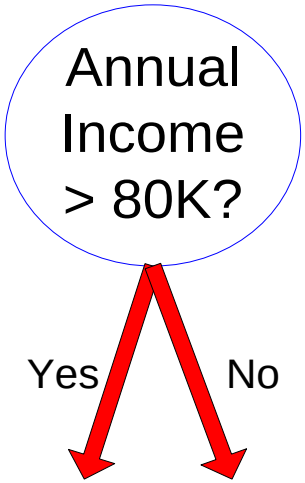


Test Condition for Ordinal Attributes

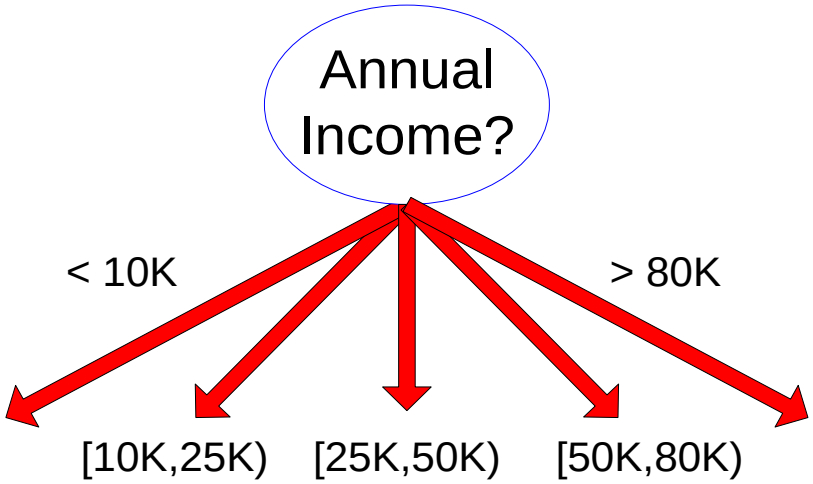
- Multi-way split:
  - Use as many partitions as distinct values
- Binary split:
  - Divides values into two subsets
  - Preserve order property among attribute values



Test Condition for Continuous Attributes



(i) Binary split

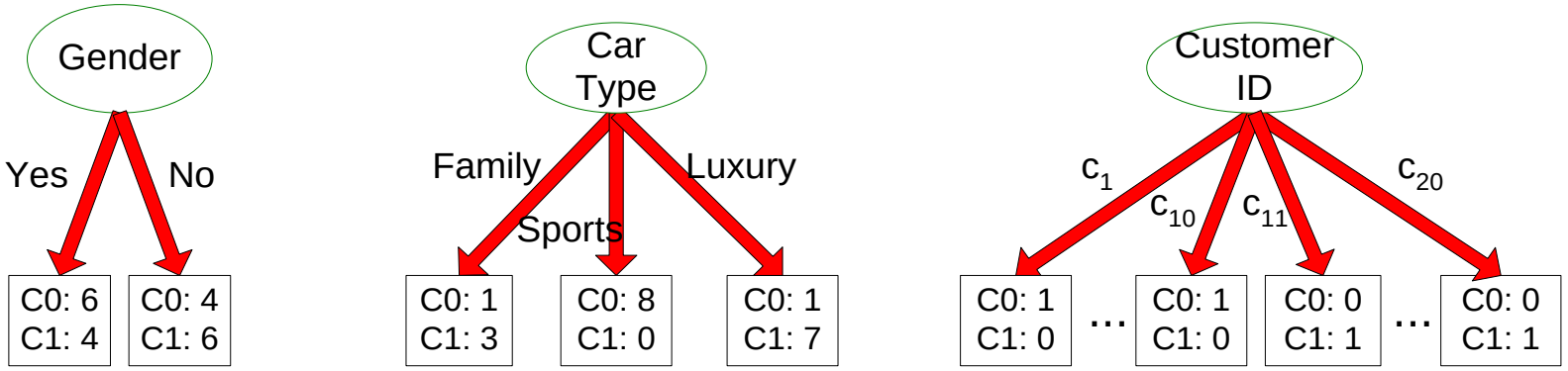


(ii) Multi-way split

How to determine the Best Split

Before Splitting: 10 records of class 0,
10 records of class 1

Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1



Which test condition is the best?

How to determine the Best Split

- Greedy approach:
  - Nodes with purer class distribution are preferred

- Need a measure of node impurity:

C0: 5 C1: 5	C0: 9 C1: 1
High degree of impurity	Low degree of impurity

Measures of Node Impurity

- Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $\boldsymbol{p_i(t)}$ is the frequency of class $\boldsymbol{i}$ at node t, and $\boldsymbol{c}$ is the total number of classes

- Entropy

$$Entropy = -\sum_{i=0}^{c-1} p_i(t) log_2 p_i(t)$$

- Misclassification error

$$Classification\ error = 1 - \max[p_i(t)]$$

Finding the Best Split

1. Compute impurity measure (P) before splitting
2. Compute impurity measure (M) after splitting
   - Compute impurity measure of each child node
   - M is the weighted impurity of child nodes
3. Choose the attribute test condition that produces the highest information gain

Gain = P - M

or equivalently, lowest impurity measure after splitting (M)

Gini Index

Gini Index

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_i(t)$ is the frequency of class i at node t, and c is the total number of classes

- Lower the Gini index, higher the homogeneity of nodes

Gini Index-Example

Instance Number	X	Y	Z	Class
1	1	1	1	A
2	1	1	0	A
3	0	0	1	B
4	1	0	0	B

The frequencies of the two output classes are given as follows:

- A = 2 (Instances 1,2)
- B = 2 (Instances 3,4)

Gini Index-Example

Attribute 'X':

- There are two possible values of X, i.e., 1, 0
- For X = 1, there are 3 instances namely instance 1, 2 and 4.
- The first two instances are labelled as class A and the third instance, i.e, record number 4 is labelled as class B.

- For X = 0, there is only 1 instance, i.e, instance number 3 which is labelled as class B.

- Computing the Gini Index for each case,
- G (X1) = 1 - (2/3) 2 - (1/3) 2 = 0.44444
- G (X0) = 1 - (0/1) 2 - (1/1) 2 = 0

Gini Index-Example

- Weighted Gini Index=probability for X having value 1 * G(X1) +
  probability for X having value 0 * G(X0)

- Probability for X having value 1= (Number of instances for X having value 1/Total number of instances) = 3/4
- Probability for X having value 0 = (Number of instances for X having value 0/Total number of instances) = 1/4

- Weighted Gini Index for two subtrees = (3/4) G(X1) + (1/4) G(X0)
- = 0.333333+0 = **0.333333**

Gini Index-Example

Atrribute Y:

- G(Y1) = 1 - (2/2) 2 - (0/2) 2 = 0
- G(Y0) = 1 - (0/2) 2 - (2/2) 2 = 0
- Weighted Gini Index = (2/4) G(Y1) + (2/4) G(Y0)

  = 0 + 0 = **0**

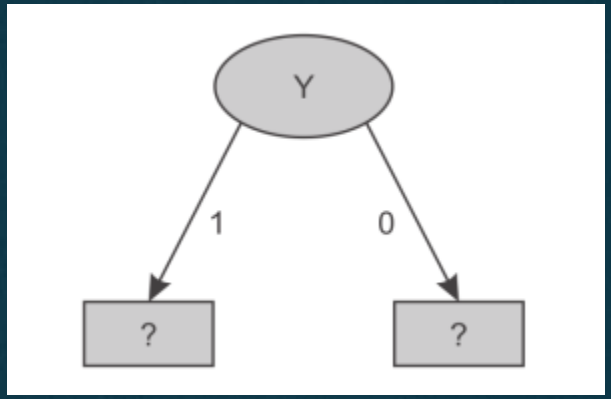
Attribute ‘Z’

- G (Z1) = 1 - (1/2) 2 - (1/2) 2 = 0.5
- G(Z0) = 1 - (1/2) 2 - (1/2) 2 = 0.5
- Weighted Gini Index= (2/4) G(Z1) + (2/4) G(Z0)

  = 0.25 + 0.25 = **0.50**

Gini Index-Example

- Since the minimum weighted Gini Index is provided by the attribute 'Y' it is used for the split



- There are two possible values for attribute Y, i.e., 1 or 0, and so the dataset will be split into two subsets based on distinct values of Y attribute

Gini Index-Example

Dataset for Y '1'

Instance Number	X	Z	Class
1	1	1	A
2	1	0	A

- When Y is 1, we get homogeneous class label, A i.e least impurity, that implies When Y is 1, class label is A
- (This can be confirmed by computing Gini index of data set i.e

  $1-(2/2)^2-(0/2)^2=0$)

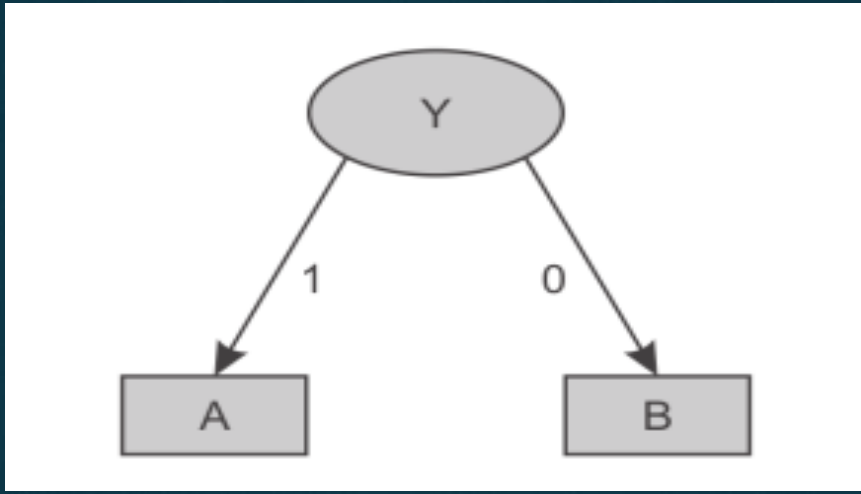
Gini Index-Example

Dataset for Y '0'

Instance Number	X	Z	Class
3	0	1	B
4	1	0	B

- Here, for Y having value 0, both records are classified as class 'B' irrespective of value of X and Z.
- (This can be confirmed by computing Gini index of data set i.e $1-(0/2)^2-(2/2)^2=0$)

Gini Index-Example



Example 2:

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Gini Index-Example

Outlook	*Temperature*	*Humidity*	*Windy*	*Play*
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

Gini Index-Example

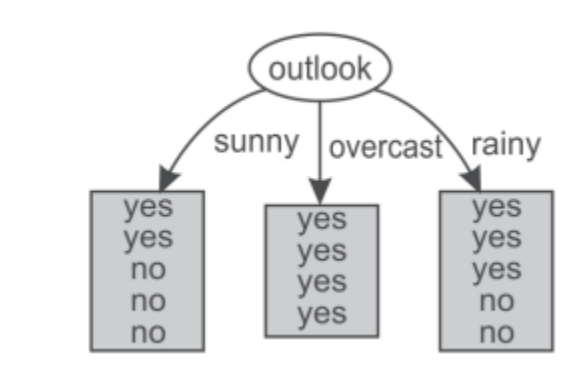
- Attribute 'Outlook'

$$G(\text{Sunny}) = G(S) = 1 - (2/5)^2 - (3/5)^2 = 0.48$$
$$G(\text{Overcast}) = G(O) = 1 - (4/4)^2 - (0/4)^2 = 0$$
$$G(\text{Rainy}) = G(R) = 1 - (3/5)^2 - (2/5)^2 = 0.48$$

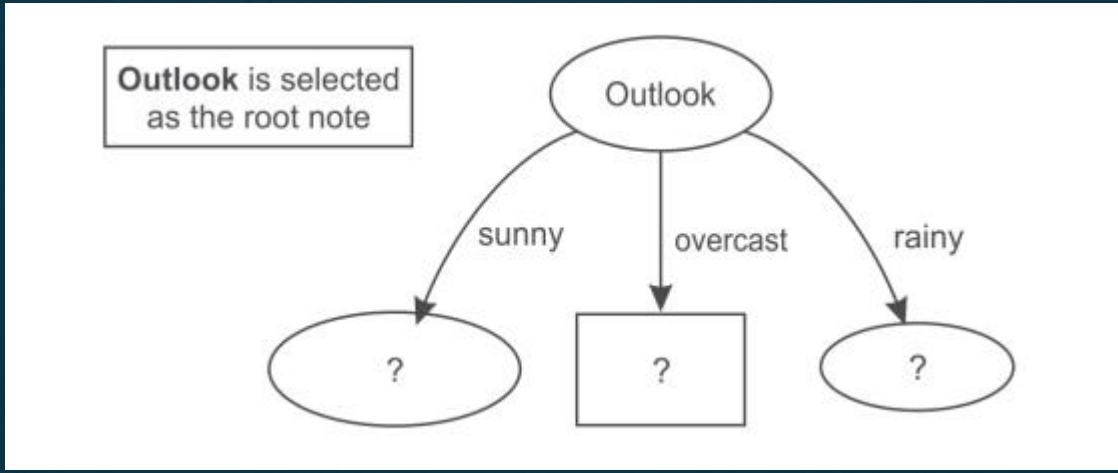
- Weighted Gini Index= $(5/14)\,G(S) + (4/14)\,G(O) + (5/14)\,G(R)$
  $= 0.171428 + 0 + 0.171428$
  $= 0.342857$

Potential Split attribute	Weighted Gini Index
Outlook	0.342857
Temperature	0.44028
Humidity	0.3673439
Windy	0.45918

Gini Index-Example



- For Outlook, as there are three possible values, i.e., sunny, overcast and rainy, the dataset will be split into three subsets based on distinct values of the Outlook attribute



Gini Index-Example

Dataset for Outlook 'Sunny'

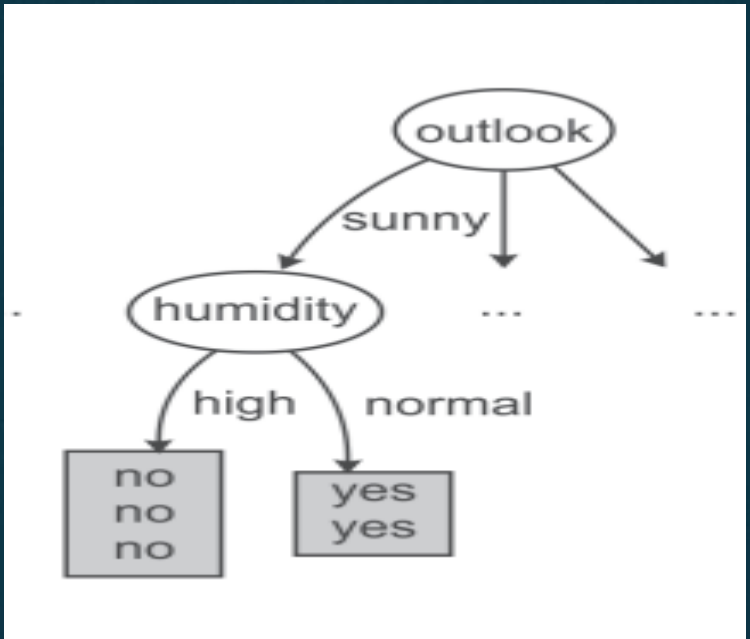
Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	No
2	hot	high	true	No
3	mild	high	false	No
4	cool	normal	false	Yes
5	mild	normal	true	Yes

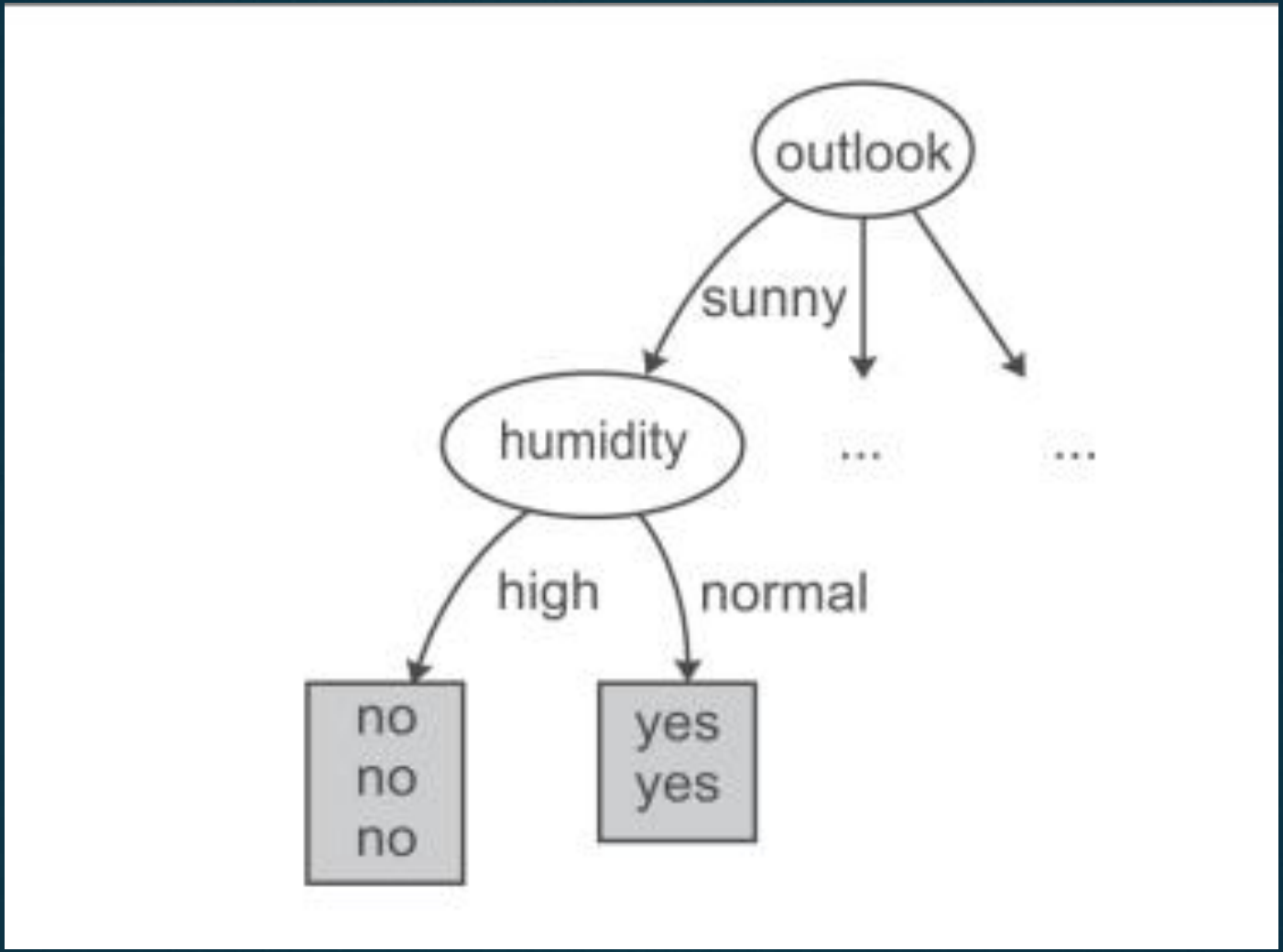
- Attribute 'Temperature'

$$G(\text{Hot}) = G(H) = 1 - (0/2)^2 - (2/2)^2 = 0$$
$$G(\text{Mild}) = G(M) = 1 - (1/2)^2 - (1/2)^2 = 0.5$$
$$G(\text{Cool}) = G(C) = 1 - (1/1)^2 - (0/1)^2 = 0$$

- Weighted Gini Index= (2/5)G(H) + (1/5) G(M) + (2/5) G(C) = 0 + 0.1 + 0 = 0.1

- Potential split Attribute — Weighted Gini index
  - Temperature — 0.1
  - Humidity — 0
  - Wind — 0.466




outlook
sunny
humidity
...
...
high
normal
no
no
no
yes
yes

Dataset for Humidity 'High'

Instance Number	Temperature	Windy	Play
1	Hot	false	No
2	Hot	true	No
3	Mild	false	No

We can clearly see that all records with high humidity are classified as play having 'No' value. On the other hand, all records with normal humidity value are classified as play having 'Yes' value.

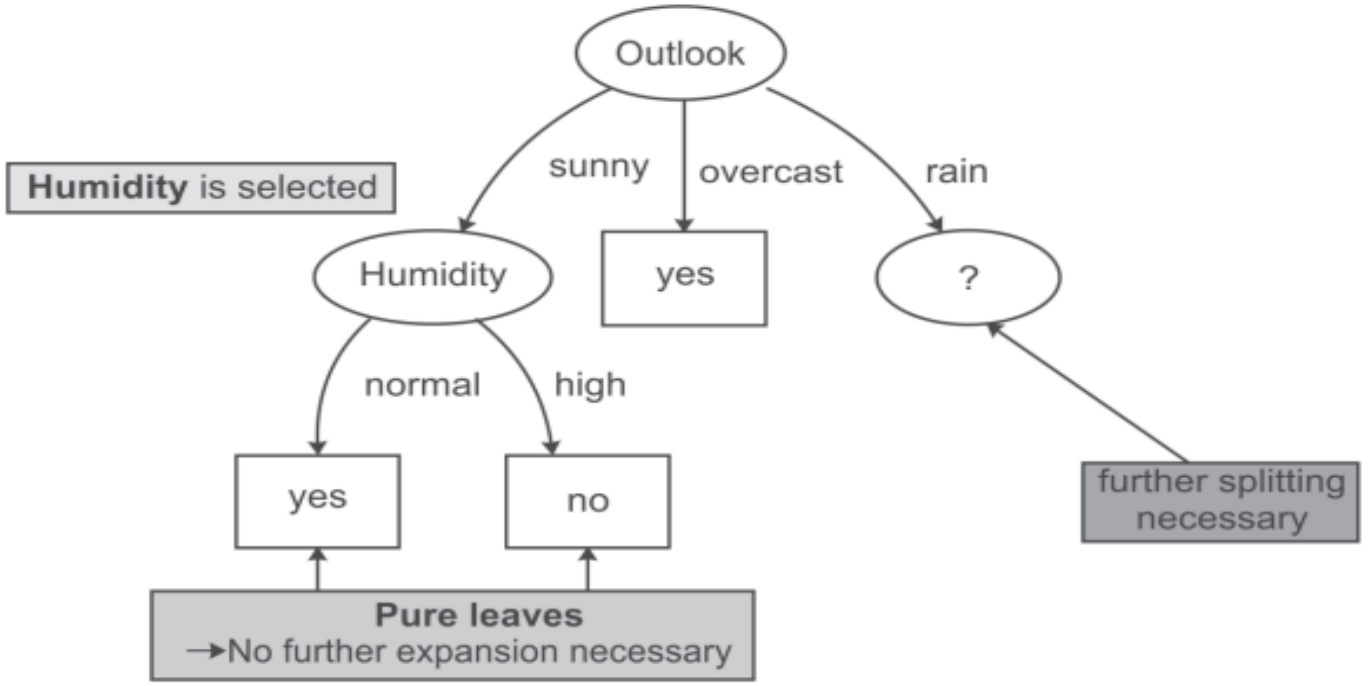
Dataset for Humidity 'Normal'

Instance Number	Temperature	Windy	Play
1	cool	false	Yes
2	mild	true	Yes

Dataset for Outlook 'Overcast'

Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

This dataset consists of 3 records. For outlook overcast all records belong to the 'Yes' class only.



Dataset for Outlook 'Rainy'

Instance Number	Temperature	Humidity	Windy	Play
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	Ttrue	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

- Potential split Attribute — Weighted Gini index
  - Temperature — 0.4666
  - Humidity — 0.4666
  - Wind — 0

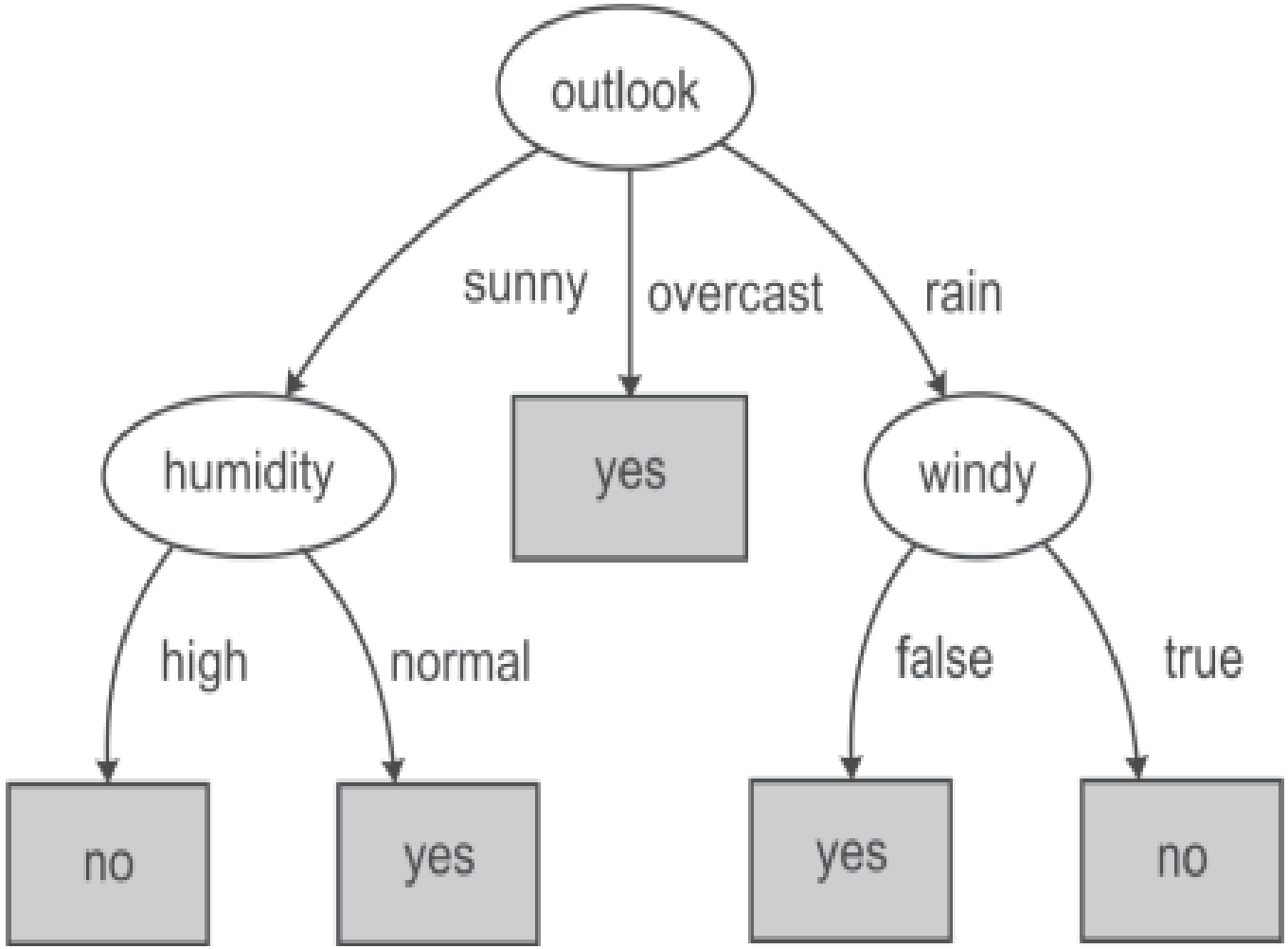
Dataset for Windy 'TRUE'

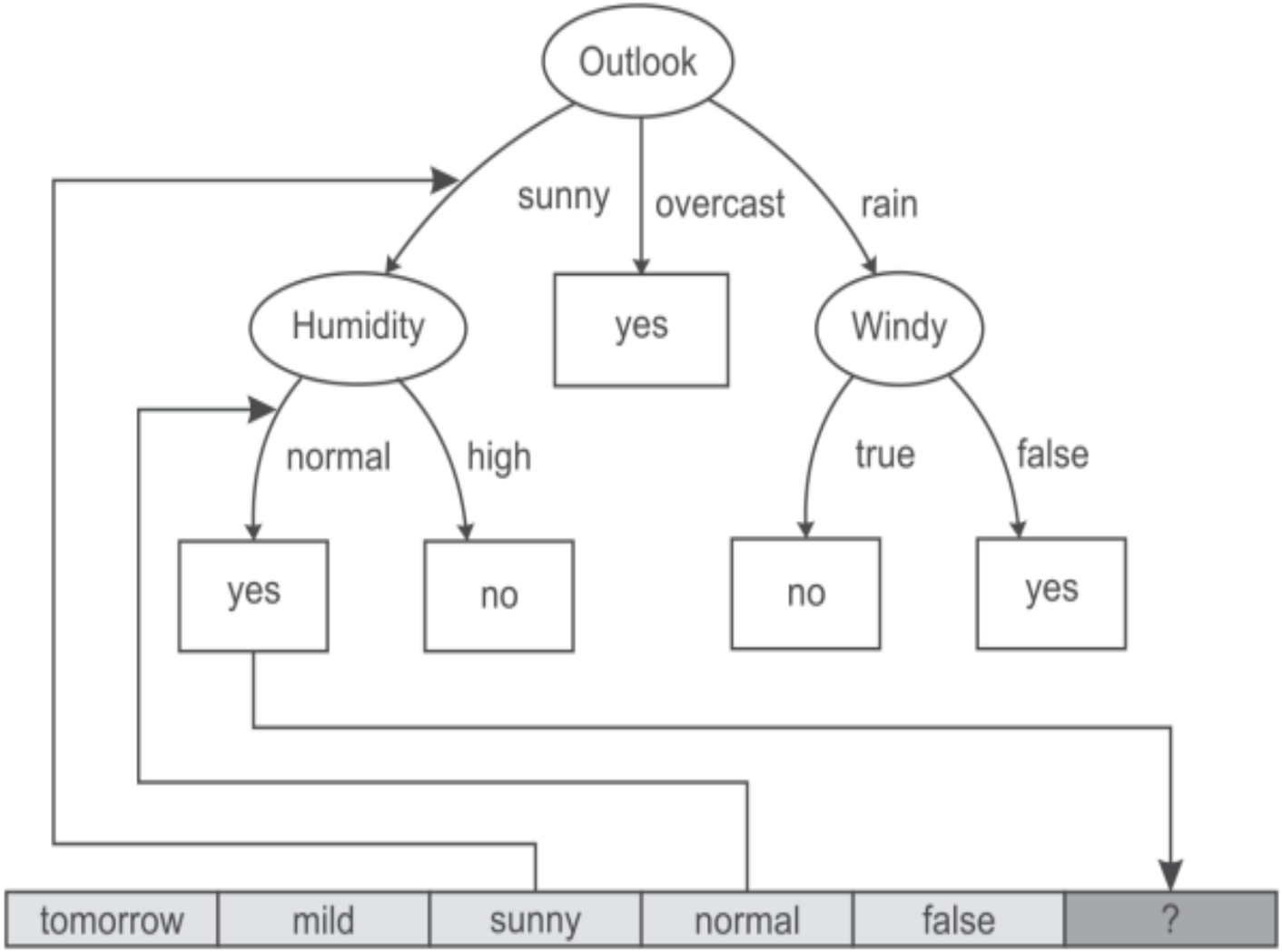
Instance Number	Temperature	Humidity	Play
1	cool	normal	No
2	mild	high	No

It is clear from the data that whenever it is windy the play is not held. Play is held whenever conditions are otherwise.

Dataset for Windy 'FALSE'

Instance Number	Temperature	Humidity	Play
1	mild	high	Yes
2	cool	normal	Yes
3	mild	normal	Yes


outlook
sunny
overcast
rain
humidity
yes
windy
high
normal
false
true
no
yes
yes
no


Outlook
sunny
overcast
rain
Humidity
yes
Windy
normal
high
true
false
yes
no
no
yes
tomorrow
mild
sunny
normal
false
?

Best split using Entropy and Information gain

Instance Number	*Outlook*	*Temperature*	*Humidity*	*Windy*	*Play*
1	sunny	hot	high	false	No
2	sunny	hot	high	true	No
3	overcast	hot	high	false	Yes
4	rainy	mild	high	false	Yes
5	rainy	cool	normal	false	Yes
6	rainy	cool	normal	true	No
7	overcast	cool	normal	true	Yes
8	sunny	mild	high	false	No
9	sunny	cool	normal	false	Yes
10	rainy	mild	normal	false	Yes
11	sunny	mild	normal	true	Yes
12	overcast	mild	high	true	Yes
13	overcast	hot	normal	false	Yes
14	rainy	mild	high	true	No

Outlook	Temp.	Humidity	Windy	Play
sunny = 5	hot = 4	high = 7	true = 6	yes = 9
overcast = 4	mild = 6	normal = 7	false = 8	no = 5
rainy = 5	cool = 4			

- In the dataset, there are 14 samples and two classes for target attribute 'Play', i.e., Yes or No. The frequencies of these two classes are: Yes = 9 No = 5
- Entropy of the whole dataset on the basis of whether play is held or not is given by:

- E = - probability for play being held * log (probability for play being held) - probability for play not being held * log (probability for play not being held)

- Probability for play having value Yes = (Number of instances for Play is Yes/Total number of instances) = 9/14
- Probability for play having value No = (Number of instances for Play is No/Total number of instances) = 5/14
- Therefore, E = - (9/14) log (9/14) – (5/14) log (5/14) = 0.9435142

- Now Let us consider each attribute one by one as split attributes and calculate the information for each attribute

Attribute 'Outlook':

- E (Sunny) = E (S) = - (2/5) log (2/5) - (3/5) log(3/5) = 0.97428
- E (Overcast) = E (O) = - (4/4) log(4/4) - (0/4) log(0/4) = 0
- E (Rainy) = E (R) = - (3/5) log(3/5) - (2/5) log(2/5) = 0.97428
- Total Entropy for these three sub-trees = probability for outlook Sunny * E(S) + probability for outlook Overcast * E(O) + probability for outlook Rainy * E(R)
- = (5/14) E(S) + (4/14) E(O) + (5/14) E(R)
- = 0.3479586 + 0.3479586
- = 0.695917

Attribute 'Temperature'

- E (Hot) = E (H) = - (2/4) log(2/4) - (2/4) log(2/4) = 1.003433
- E (Mild) = E (M) = - (4/6) log(4/6) - (2/6) log(2/6) = 0.9214486
- E (Cool) = E (C) = - (3/4) log(3/4) - (1/4) log(1/4) = 0.814063501
- Total Entropy for these three sub-trees = (4/14) E(H) + (6/14) I(M) + (4/14) E(C)
- = 0.2866951429 + 0.3949065429 + 0.23258957 = 0.9141912558

Attribute 'Humidity'

- E (High) = E (H) = - (3/7) log(3/7) - (4/7) log(4/7) = 0.98861
- E (Normal) = E (N) = - (6/7) log(6/7) - (1/7) log(1/7) = 0.593704
- Total Entropy for the two sub-trees = (7/14) E(H) + (7/14) E(N)

  = 0.494305 + 0.29685 = 0.791157

Attribute 'Windy'

- E (True) = E (T) = - (3/6) log(3/6) - (3/6) log(3/6) = 1.003433
- I (False) = I (F) = - (6/8) log(6/8) - (2/8) log(2/8) = 0.81406
- Total information for the two sub-trees = (6/14) E(T) + (8/14) E(F)

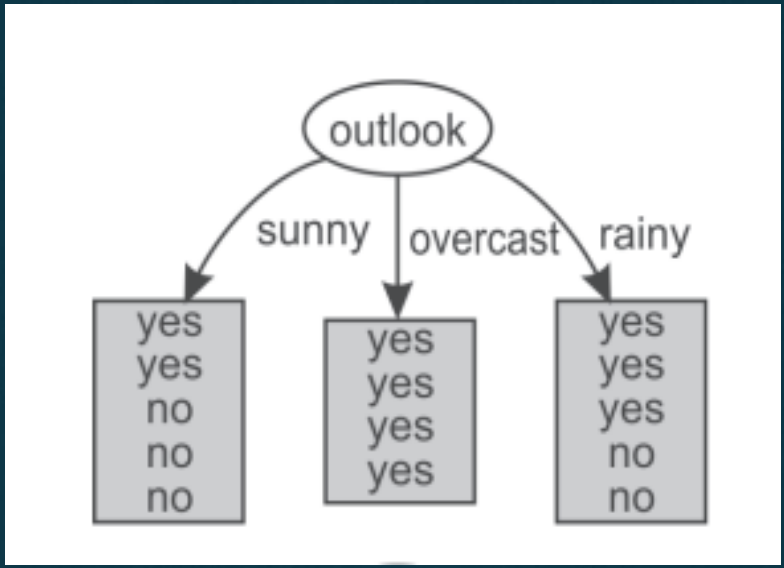
  = 0.4300427 + 0.465179 = 0.89522

- The information gain can now be computed:

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Outlook	0.9435	0.6959	0.2476
Temperature	0.9435	0.9142	0.0293
Humidity	0.9435	0.7912	0.15234
Windy	0.9435	0.8952	0.0483

- From the above table, it is clear that the largest information gain is provided by the attribute 'Outlook' so it is used for the split

- For Outlook, as there are three possible values, i.e., Sunny, Overcast and Rain, the dataset will be split into three subsets based on distinct values of the Outlook attribute



Dataset for Outlook 'Sunny'

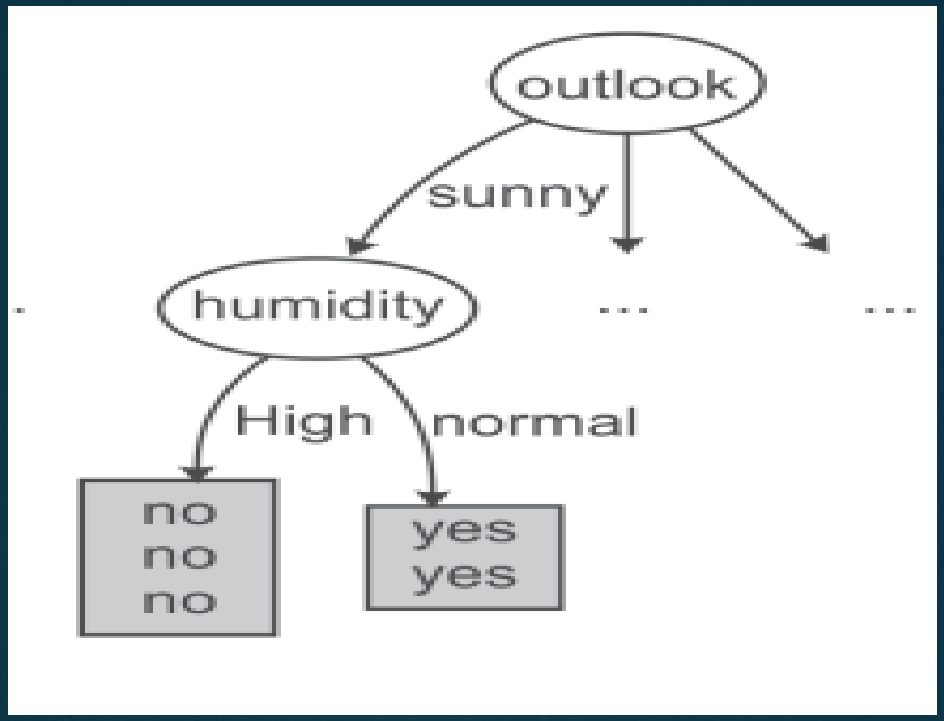
Instance Number	Temperature	Humidity	Windy	Play
1	Hot	high	false	No
2	Hot	high	true	No
3	Mild	high	false	No
4	Cool	normal	false	Yes
5	Mild	normal	true	Yes

- Entropy of the whole dataset on the basis of whether play is held or not is given by I = - probability for Play Yes * log (probability for Play Yes) - probability for Play No * log (probability for Play No)
- E = (-2/5) log (2/5) – (3/5) log(3/5) = 0.97

- Let us consider each attribute one by one as a split attribute and calculate the Entropy for each attribute.

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Temperature	0.97	0.40137	0.56863
Humidity	0.97	0	0.97
Windy	0.97	0.954239	0.015761

- The largest information gain is provided by the attribute 'Humidity' and it is used for the split



- There are two possible values of humidity so data is split into two parts, i.e. humidity 'high' and humidity 'low'

Dataset for Humidity 'High'

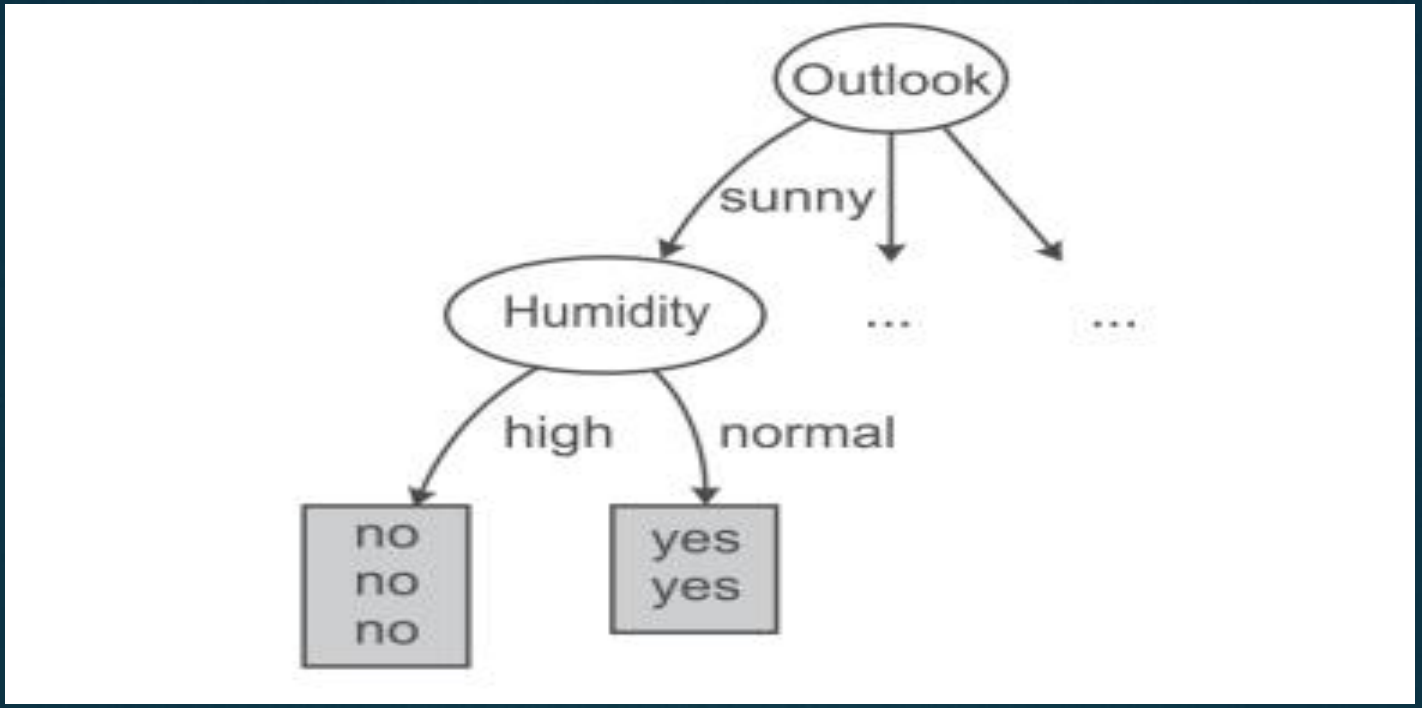
Instance Number	Temperature	Windy	Play
1	hot	false	No
2	hot	true	No
3	mild	false	No

- As the dataset represents the same class 'No' for all the records, therefore for Humidity value 'High' the output class is always 'No'

Dataset for Humidity 'Normal'

Instance No	Temperature	Windy	Play
1	cool	false	Yes
2	mild	true	Yes

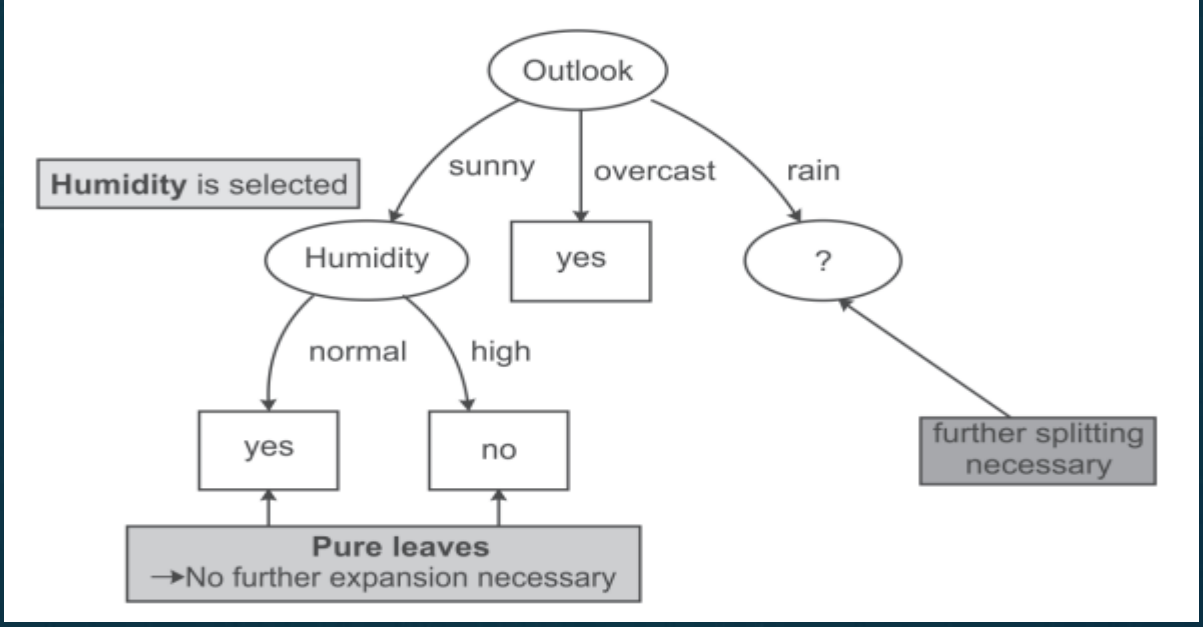
- When humidity's value is 'Normal' the output class is always 'Yes'


Outlook
sunny
Humidity
...
...
high
normal
no
no
no
yes
yes

- Dataset for Outlook 'Overcast'

Instance Number	Temperature	Humidity	Windy	Play
1	hot	high	false	Yes
2	cool	normal	true	Yes
3	mild	high	true	Yes

- For outlook Overcast, the output class is always 'Yes'.
- it has been concluded that if the outlook is 'Overcast' then play is always held



- Dataset for Outlook 'Rainy'

Instance Number	*Temperature*	*Humidity*	*Windy*	*Play*
1	Mild	High	False	Yes
2	Cool	Normal	False	Yes
3	Cool	Normal	True	No
4	Mild	Normal	False	Yes
5	Mild	High	True	No

- Information of the whole dataset= (-3/5) log (3/5) – (2/5) log (2/5) = 0.97428

- Let us consider each attribute one by one as split attributes and calculate the information provided for each attribute.

Potential Split attribute	*Information before split*	*Information after split*	*Information gain*
Temperature	0.97428	0.954297	0.019987
Humidity	0.97428	0.9512	0.02308
Windy	0.97428	0	0.97428

- Hence, the largest information gain is provided by the attribute 'Windy' and this attribute is used for the split.

Dataset for Windy 'TRUE'

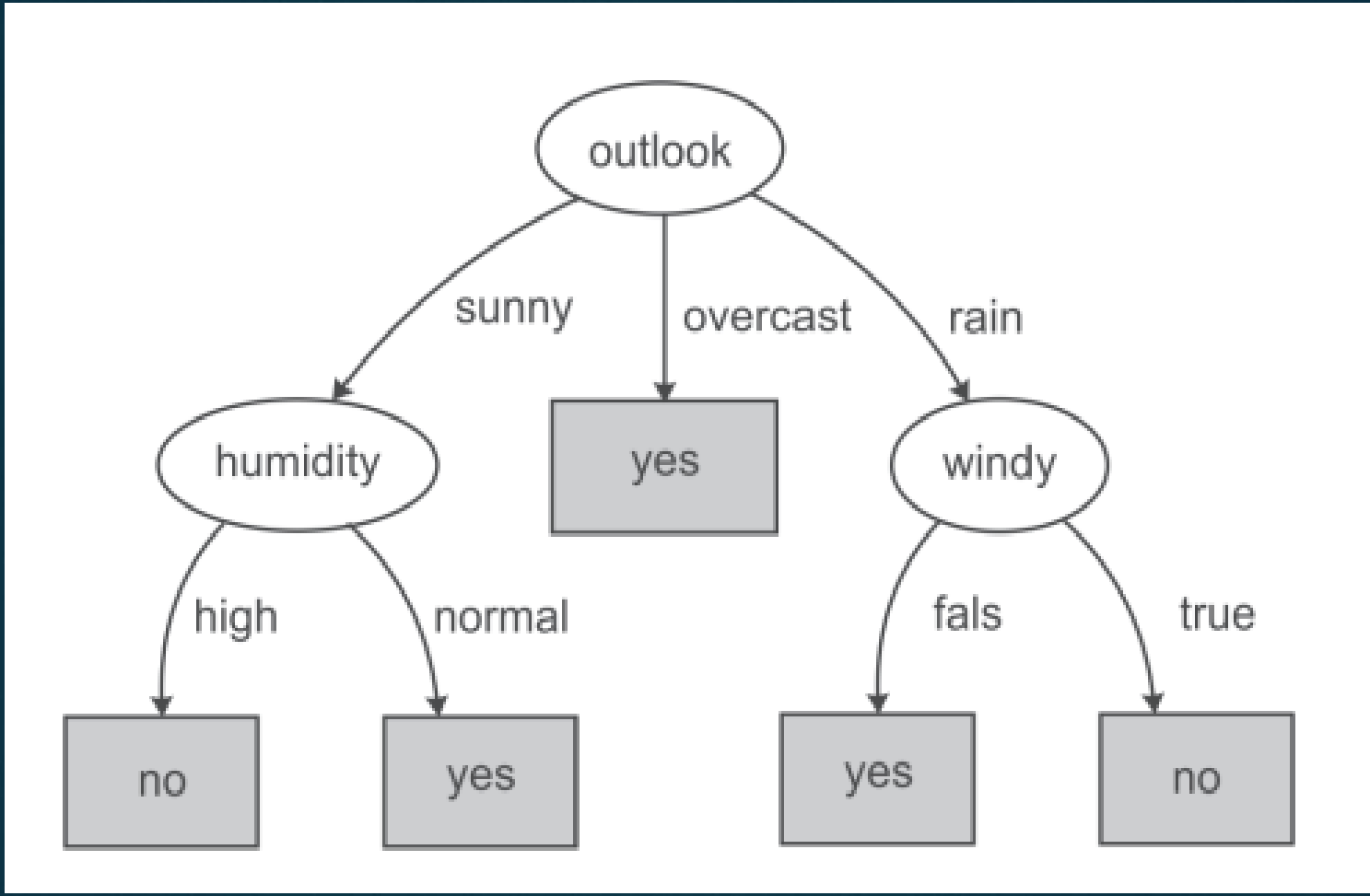
Instance Number	Temperature	Humidity	Play
1	cool	normal	No
2	mild	high	No

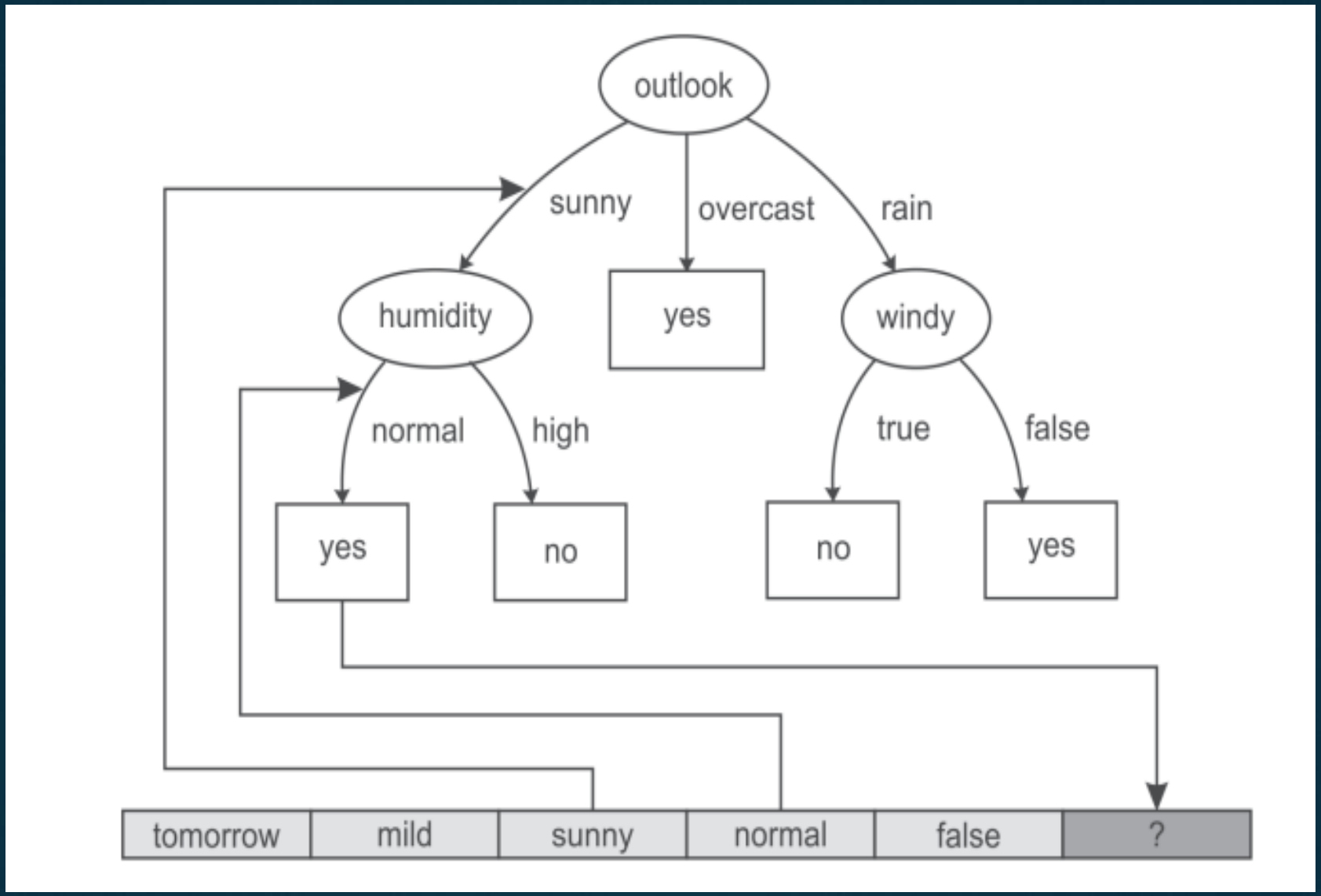
- From above table it is clear that for Windy value 'TRUE', the output class is always 'No'

Dataset for Windy 'FALSE'

Instance Number	Temperature	Humidity	Play
1	Mild	High	Yes
2	Cool	Normal	Yes
3	Mild	Normal	Yes

- Also for Windy value 'FALSE', the output class is always 'Yes'


outlook
sunny
overcast
rain
humidity
yes
windy
high
normal
fals
true
no
yes
yes
no


outlook
sunny
overcast
rain
humidity
yes
windy
normal
high
true
false
yes
no
no
yes
tomorrow
mild
sunny
normal
false
?